Stack Canary Protection and PMP-Based Stack Guards in S3K

Agaton Pirander Oliver Marcusson

{agpira, olmar}@kth.se

October 2026

Abstract

A stack buffer overflow in a C program can overwrite saved return state and take over control flow. Process separation in a kernel such as S3K does not prevent this, because the corruption happens inside the process. We add compiler-generated stack canaries to the S3K kernel and its applications, and extend them with virtualized PMP stack guards: the stack of the protected application has no permanent memory protection entry, a trusted monitor maps small guard-free windows on demand, and the canary and saved `ra`/`s0` of every live frame are never mapped. An overflow therefore faults at the first write to a guard, and the isolated monitor suspends the application. Against an unprotected build, a 137-byte input redirects control flow to a target function. With protection, the target is not reached, and a write past the top of the stack is stopped by the monitor. The main limitation is that the monitor trusts the frame events the application sends.

1 Introduction

Embedded and kernel software is still mostly written in C, and C does not check array bounds. A function that copies attacker-controlled input into a fixed-size stack buffer without a length check can write past the buffer into the data stored above it. On RISC-V, that data includes the saved return address `ra` and the saved frame pointer `s0`. Overwriting the saved `ra` lets an attacker choose where the function returns, which is enough to hijack control flow.

S3K is a capability-based separation kernel for RISC-V [4]. It isolates processes from each other with Physical Memory Protection (PMP), so a compromised process cannot directly read or write the memory of another. Separation does not, however, help inside a process: the overflow, the corrupted frame and the hijacked return all happen in memory the process is allowed to write. S3K is a useful setting for this problem because it is small enough to change in a course project, it has no virtual memory, and its monitor capabilities let one trusted process manage the PMP configuration of another.

Research question. How can compiler-generated stack canaries and capability-controlled memory protection detect and contain stack corruption in S3K, and what practical constraints arise when implementing them?

Scope and contributions. We integrate GCC stack canaries into the S3K kernel and its user applications, and build a vulnerable demonstration application with a reproducible exploit. On top of the canaries we implement virtualized PMP stack guards: a monitor process that treats the eight PMP entries of the application like a translation lookaside buffer. It maps stack windows on demand and never maps a guard, so that an overflow is stopped before it overwrites anything. Individual responsibilities are described in Section 7.

2 Threat Model and Countered Vulnerabilities

2.1 Threat model

The system consists of three S3K processes (Table 1). The attacker controls the input to `vuln` over the UART, and `vuln` reads it with `alt_gets`, which has no bounds check. The attacker can exploit any memory-safety bug reachable from that input. The kernel, the boot process `app0` and the monitor are trusted. `vuln` is a benign application that is being exploited, not a malicious program: its code is

instrumented at build time, and we assume that the frame events it sends to the monitor are correct as long as its control flow has not been taken over. The goal of the design is to stop the corruption before that happens.

Process	PID	Role
`app0`	0	Boot process. Grants memory, time, PMP, monitor and socket capabilities, then starts the monitor. Trusted.
`monitor`	1	Owns the guard table, maps stack windows into `vuln`'s PMP, classifies faults and stops `vuln` on an attack. Trusted.
`vuln`	2	Instrumented application with the unsafe `alt_gets` call. Sends frame events and faults to the monitor. Untrusted input.

Table 1: Processes in the `projects/canaries` system.

2.2 Countered vulnerabilities

The design targets stack-based memory corruption in `vuln`:

- *Linear stack-buffer overflow* into the canary and the saved `ra`/`s0`, the classic control-flow hijack.
- *Non-linear writes* that skip the canary, such as an indexed write, and hit the saved return state directly.
- *Canary disclosure*, where the attacker reads the canary and writes the correct value back while overwriting `ra`.
- *Overflows through a pointer* to a buffer in a frame further up the call chain.
- *Code injection on the stack*, where the payload itself is executed.
- *Stack exhaustion*, where the stack grows past its bounds.

2.3 Non-goals

The design does not protect data-only attacks that stay inside a single frame and never touch a guard, such as overwriting a neighbouring local variable. It does not protect memory outside the stack, such as the heap, `.data` or `.bss`. Kernel and hardware attacks, side channels and denial of service are out of scope.

3 Background

S3K runs on RISC-V and uses capabilities for every resource: memory ranges, time slices, IPC channels and sockets, and *monitor* capabilities that let one process suspend, resume and reconfigure another. A memory capability can be turned into a PMP capability and loaded into one of the process's hardware PMP entries. On our platforms (`qemu_virt`) each process has `S3K_PMP_CNT` = 8 entries. S3K PMP capabilities use NAPOT encoding, so every region is a power of two in size and aligned to its size, with a minimum of 8 bytes. A user-mode access that matches no PMP entry raises an access fault, which S3K delegates to the process's own trap handler.

GCC's `-fstack-protector-strong` instruments every function that has a local array or a local whose address is taken. The prologue copies the global `__stack_chk_guard` into the frame, between the local buffers and the saved registers. The epilogue compares the copy with the global value and calls `__stack_chk_fail` if they differ. A canary detects a linear overwrite only when the function returns, after the damage is done.

On systems with virtual memory, a guard page is an unmapped page next to a protected region, so that any access to it faults. S3K has no page tables and only eight PMP entries, far too few to give every stack frame its own guard. Our design solves this by using the PMP entries as a cache over a complete guard table kept in software, in the same way that a translation lookaside buffer caches a page table.

Draft prompt: Keep the next paragraph only if the course requires citing these papers; otherwise delete it and the three bibliography entries.

Larsen et al. [1] classify defenses based on unpredictability. A random canary is such a secret, but it is not layout diversification, and it fails as soon as it is disclosed. This is the gap our guards address. Patagonix [2] motivates monitoring from outside the compromised component. Our monitor plays a similar role, but as an S3K process with monitor capabilities, not a hypervisor. C-FLAT [3] attests the runtime control flow of embedded software. Our design does not attest control flow; it prevents one class of corruption that leads to control-flow hijacking.

4 Design and Implementation

4.1 Architecture

All code is in `projects/canaries`. `app0` boots, splits memory between the other two processes, and creates the capabilities in Table 2. It sets `vuln`'s program counter but does not resume it. The monitor installs `vuln`'s trap handler and trap stack through its monitor capability, then resumes `vuln` and enters its main loop. Both message sources in `vuln`, the instrumentation hooks and the trap handler, share one socket on channel 0, and `data[0]` of every message names its kind (Table 3).

PMP slot	Region	Access	Contents
0	`0x80030000`, 32 KiB	RWX	Code, data, `.bss`, and the pinned trap region at `0x80037000` (handler and 2 KiB trap stack). Permanent.
1	UART, 8 bytes	RW	Permanent.
2–7	Stack windows	RW	Loaded and evicted by the monitor.
—	`[0x8003F000, 0x80040000)`	none	The 4 KiB stack. No permanent PMP entry.

Table 2: Memory layout and PMP slots of `vuln`. The range between the image and the stack is never mapped.

`data[0]`	Sent by	`data[1]`	`data[2]`	`data[3]`
`MSG_GUARD_PUSH` = 1	enter hook	canary address	frame (top of frame)	function address
`MSG_GUARD_POP` = 2	exit hook	frame	—	—
`MSG_FAULT` = 3	trap handler	`ECAUSE`	`EVAL`	`EPC`

Table 3: IPC messages from `vuln` to the monitor. The function address is used only for logging.

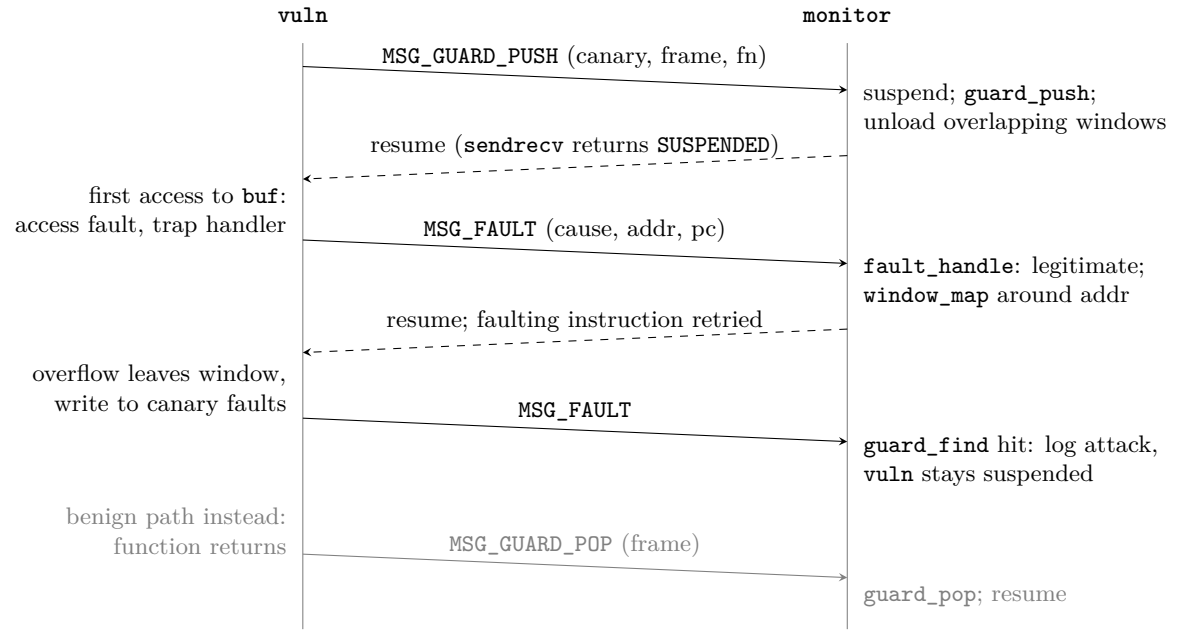


Figure 1: Lifecycle of a guarded stack frame. Every message blocks `vuln` until the monitor resumes it.

4.2 Design choices and rejected alternatives

Guards in addition to canaries. A canary alone detects a linear overwrite when the function returns, after the saved `ra` has already been replaced, and the response (`__stack_chk_fail`) runs inside the compromised process. It misses writes that skip the canary, and it fails if the canary is leaked. We keep the canaries as the baseline and add guards that stop the first access to the canary or the saved return state, before anything is overwritten, with the response handled by an isolated process.

Per-frame guards instead of one stack guard. A single unmapped region below the stack only catches stack exhaustion. To catch an overflow inside the stack, every live frame needs its own guards. We register two per frame: `[canary, canary+8)` and the saved `ra`/`s0` at `[frame-16, frame)`.

Living with eight PMP entries. Two entries are permanent (image and UART), which leaves six for the stack. A heap-protection project on S3K rejected guard pages for this reason. We avoid the limit by inverting the question: guards are not PMP entries at all. The guard table is ordinary monitor memory (64 frames, 128 guards, in the evaluated build), and the six PMP entries only describe which guard-free windows are currently mapped. When all six are in use, the monitor evicts one round-robin. Running out of PMP entries therefore costs extra faults, not protection.

A separate monitor. The guard table and the PMP configuration must be out of reach of the attacker. S3K only lets a process change another process's PMP through a monitor capability, so the monitor is a separate process with a monitor capability over `vuln`, and `vuln` holds no capability that could change its own stack mappings.

4.3 Compiler-generated stack canaries

Both the kernel and every application are built with `-fstack-protector-strong`. Each provides its own `__stack_chk_guard` and `__stack_chk_fail`. The kernel seeds its guard in `stack_chk_guard_init`, which `head.S` calls after clearing `.bss` and before `kernel_init`; the function is compiled without a stack protector because it changes the value it would check. The seed mixes the timer, the guard's own address and the hart ID, then applies the MurmurHash3 finalizer (multiplication by `0xff51afd7ed558ccd` and a 33-bit xor-shift). Applications call `util_stack_protect_init` first in `main`, which does the same with `s3k_get_time`. In both cases the lowest byte is cleared, so a string copy that stops at a NUL byte cannot reproduce the canary.

Figure 2 shows the frame GCC 15.1 generates for `greet` in the protected build. The 128-byte buffer starts at `sp+8` and ends exactly at the canary, so the canary is the first thing a linear overflow reaches. The callee-saved `s1` sits between the canary and `s0`, so the distance from the canary to `ra` depends on how many registers a function saves. The guards therefore cannot assume a fixed canary offset.

Offset	Contents	Guard
`sp+168`	saved `ra`	return guard `[frame-16, frame)`
`sp+160`	saved `s0`	return guard
`sp+152`	saved `s1`	—
`sp+136`	canary (`sd a5,136(sp)`)	canary guard `[canary, canary+8)`
`sp+8`	`char name[128]`	mapped on demand

Figure 2: Stack frame of `greet` (176 bytes, frame = `sp+176`) in the protected build, read from the disassembly of `vuln.elf`.

A canary mismatch ends in `__stack_chk_fail`. The kernel version prints a message and halts the hart in a `wfi` loop. The application version prints `*** stack smashing detected ***` and spins forever. Spinning is not termination: the process keeps its time slice and capabilities. In the full design this matters less, because an overflow faults on the guard before the canary check is ever reached, and the monitor suspends `vuln`.

4.4 Virtualized PMP stack guards

Frame registration. `vuln` is compiled with `-finstrument-functions`, so GCC calls `__cyg_profile_func_enter` after every prologue and `__cyg_profile_func_exit` before every

epilogue. The canary is always written before the enter hook runs, so registering the guard never interferes with the prologue. The enter hook is a short assembly preamble that captures the stack pointer of the instrumented function before the hook opens its own frame. The C part adds the function's frame size, taken from a table that the build derives from the `-fstack-usage` output, to get the top of the frame, locates the canary, and sends `MSG_GUARD_PUSH`. The exit hook sends `MSG_GUARD_POP`. At `-Os`, GCC tail-calls the exit hook after the canary check, so the epilogue would read the canary while its guard is still registered, and every normal return would be reported as an attack. `vuln` is therefore built with `-fno-optimize-sibling-calls`, which moves the exit hook before the canary load in the epilogue.

Draft prompt: Agaton: confirm how the hook locates the canary and frame (`stack_usage.c`).

Resume is the reply. S3K only lets a monitor change the PMP of a process that is suspended. A process blocked in IPC is not enough: `s3k_mon_pmp_unload` on it returns `S3K_ERR_INVALID_STATE`. The monitor therefore never sends a reply message. `vuln` calls `s3k_sock_sendrecv` and blocks; the monitor receives the message, suspends `vuln`, updates the guard table and the PMP, and resumes it. `vuln`'s `sendrecv` then returns `S3K_ERR_SUSPENDED`, which means the request is done. On an attack, the monitor does not resume `vuln`. The socket is `S3K_IPC_NOYIELD`: with `S3K_IPC_YIELD`, the waiting client timed out at the end of its time slice and ran on before the monitor had finished.

Trap handler. The handler lives in a `.trap.text` section that the linker script places at `0x80037000`, and its 2 KiB trap stack ends at `0x80038000`. Both are inside the permanent image mapping, so the handler can never fault on its own stack. It is marked `no_instrument_function` so it does not send frame events, and `used` so link-time optimization cannot drop it. It sends `MSG_FAULT` with `ECAUSE`, `EVAL` and `EPC`, retries while the monitor is not yet receiving, and returns. Returning restores `EPC`, which points at the faulting instruction, so once a window is mapped the instruction runs again and succeeds.

Fault classification. `fault_handle` applies the checks in order and stops at the first match:

1. A cause other than an instruction, load or store access fault (`ECAUSE` 1, 5, 7) is a crash.
2. An instruction fetch from the stack is an attack. Windows are read-write only, so stack shellcode can never run.
3. An address inside a registered guard is an attack. The log names the guard kind and the owning function.
4. An address outside `[0x8003F000, 0x80040000)` is out of bounds, which covers both stack exhaustion and a write past the top of the stack.
5. The same `(EVAL, EPC)` pair twice in a row is a fault loop.
6. Otherwise the access is legitimate: the monitor maps a window and resumes `vuln`.

Every stop logs the reason, cause, address and program counter and leaves `vuln` suspended.

Window mapping and invalidation. `window_map` searches from the whole 4 KiB stack downward in powers of two for the largest NAPOT block that contains the faulting address, lies inside the stack and overlaps no guard. A buffer that ends at its canary and is aligned to its size is covered by a single window. The monitor derives a PMP capability for that block from its memory capability over the stack, moves it into `vuln`'s capability table, and loads it into a free slot between 2 and 7, evicting one round-robin if all six are in use. Eviction moves the capability back and deletes it. When a new guard is registered, every loaded window that overlaps it is evicted, so no window ever covers a live guard.

Implementation status. *Draft prompt: Replace `<commit>` with the evaluated commit, and confirm that every component above is in it. At `ab70873` plus the uncommitted `windows.c/faults.c` work, the enter and exit hooks still send `(function, caller, sp)` instead of `(canary, frame, function)`, so the monitor registers the wrong addresses and guards are not enforced.*

5 Evaluation

5.1 Experimental setup

All runs use the `qemu_virt` platform (one hart) on QEMU 8.2.2 with `-icount 3`, inside the project's `s3k-dev` container. The code is compiled with GCC 15.1.0 for `rv64imac` with `-Os -g3 -fstack-protector-strong -fstack-usage`, and `vuln` additionally with `-finstrument-functions -fno-optimize-sibling-calls`. `-flto` is disabled, because it breaks `-fstack-usage`. The unprotected build is the same tree with `-fno-stack-protector` in `projects/build.mk`. Every frame offset below is read from the disassembly of the tested `vuln.elf`, not from earlier notes. The input is sent over the UART. The suspension experiment is `projects/test.07.ipc-suspend-reply`.

Draft prompt: Add the evaluated commit hash.

5.2 Baseline results

In the unprotected build, `greet` opens a 144-byte frame with the buffer at `sp+0` and `ra` at `sp+136`. The target `win` is at `0x80030082`, whose second byte is zero, so a full address cannot pass through `alt_gets`. The saved `ra` is `0x80030176`, the return point in `main`, which already has the same upper bytes. The exploit sends 136 bytes of padding followed by `0x82`, and `alt_gets` writes the terminating NUL into the second byte. The return address becomes `0x80030082`, and the run prints `You exploited the binary!`. The exploit reads the padding, the target and the original return address from the ELF, and picks the shortest such partial overwrite.

Test	Observed result
Benign input, protected build	Both prompts answered and returned normally.
Unprotected build, 136 bytes + `0x82`	Return redirected to `win`; `You exploited the binary!` printed.
Protected build, 136 `A` (canary overwritten)	`*** stack smashing detected ***`; `vuln` spins in `__stack_chk_fail`.
Protected build, 200 `A` (past stack top)	Monitor: `out of bounds cause:7 addr:80040000 pc:80030E1C`; `vuln` left suspended.
PMP unload while client blocked	`S3K_ERR_INVALID_STATE` in 10 of 10 rounds.
PMP unload/load while suspended	Success in 10 of 10 rounds; client's `sendrecv` returned `S3K_ERR_SUSPENDED`.

Table 4: Baseline and infrastructure results.

The 200-byte run shows the bounds check working in practice. `greet`'s buffer is the last frame before the top of the stack, so the overflow reaches `0x80040000`. No PMP entry covers that address, the store inside `alt_gets` faults, and the monitor stops `vuln` before the write happens. In the 136-byte run, the input fills the 128-byte buffer and overwrites all eight bytes of the canary; the terminating NUL lands in the padding below `s1`. The epilogue's comparison fails and calls `__stack_chk_fail`, which prints its diagnostic. Attaching GDB afterwards shows `vuln`'s program counter at `0x80030a9e`, the infinite loop at the end of `__stack_chk_fail`, and `vuln` still runnable: the canary detected the overflow, but the process was not stopped. With 128 bytes, the NUL only overwrites the canary's low byte, which is already zero, and the function returns normally.

Draft prompt: Rerun the 200-byte case on the final commit; the program counter in the log line may move.

5.3 Attack matrix

Each attack runs against the canary-only build and the full design. Table 5 gives the expected outcome from the design next to the observed one.

Draft prompt: Run each row and fill in the last column with the monitor's log line or the canary diagnostic. Separate observed results from expectations in the discussion below the table.

Cost and validity. Performance was not a goal and was not measured. The cost of the design is one IPC round trip for every function call and return, and one more for every fault that maps a window.

Draft prompt: Optionally report faults per call for `greet`.

Attack	Expected, canary only	Expected, full design	Observed (canary / full)
Linear overflow into `ra`	caught at return	stopped at first guard write	*To be measured*
Leak canary, write it back	missed	stopped: the read faults	*To be measured*
Indexed write skipping the canary	missed	stopped: return guard	*To be measured*
Overflow through pointer to a frame further up	caught at return	stopped: every frame guarded	*To be measured*
Shellcode on the stack	runs	stopped: windows are not executable	*To be measured*
Deep recursion	missed	stopped: bounds check	*To be measured*

Table 5: Attack matrix.

All results come from QEMU, not real hardware, and from a small test application with two vulnerable functions.

6 Why the Design Counters the Attacks

Linear overflow. A write that runs off the end of a buffer leaves the window that covers the buffer. The next address is the canary, which is a guard and is never mapped, so the store faults and `guard_find` identifies it. Nothing past the window is written, so the saved `ra` is intact when `vuln` is stopped. The canary remains as a second line of defense.

Writes that skip the canary. An indexed write that jumps over the canary lands on the saved `s0` or `ra`, both inside the return guard `[frame-16, frame)`. This is the case a canary cannot detect.

Canary disclosure. Guards are never mapped for reading either, so the attacker cannot read the canary in order to write it back. The protection no longer depends on the canary's randomness, because the value cannot be read at all.

Frames further up. The guard table covers every live frame, not only the current one. An overflow through a pointer to a buffer in a caller's frame hits that caller's guards. Eviction does not weaken this, because guards are never mapped regardless of how many windows are in use.

Shellcode and stack exhaustion. Windows are RW only, and a fetch from the stack is classified as an attack. Stack growth below `0x8003F000`, or a write past `0x80040000`, hits memory that is never mapped and is classified as out of bounds; the 200-byte run in Table 4 shows the second case.

Attacks on the protection itself. The guard table and the window logic live in the monitor's memory, which `vuln` cannot access. `vuln`'s capabilities do not include any memory capability over its stack or any monitor capability, so it cannot map windows itself. Its only influence on the monitor is the messages it sends.

What is not stopped. Within the threat model, an attacker who never takes over control flow cannot forge frame events, because the hooks only run at real function boundaries. An attacker who does gain code execution by some other route could send `MSG_GUARD_POP` for a frame to unguard it, or skip registration. The image is mapped RWX, so a write primitive into `.data` or `.bss` could also place code there. Overwrites of locals inside a frame are not detected.

7 Individual Contributions

7.1 Oliver Marcusson

Canary integration and experimental infrastructure. I added stack canary support to the kernel and the applications (`f9ae9c4`): `-fstack-protector-strong` in the build, guard initialization before

`kernel_init` in `head.S`, and separate kernel and application handlers. Initialization had to run before the first protected function and be compiled without a protector itself, since it changes the value it would otherwise check. I created the vulnerable demonstration project (`050237d`), made `alt_gets` usable from a terminal (`ad0c336`), and put the guest UART on a TCP socket (`2973ec0`) so that the attack could be scripted. On top of that I wrote the pwntools exploit (`b89cd91`), which reads the target address and the `ra` offset from the ELF instead of hard-coding them, because every change to `greet` moves the frame. Before this I set up the shared project template and development container used by the group.

Shared interface and suspension experiment. I wrote the shared interface in `interface.h` and the `test.07.ipc-suspend-reply` experiment (`d341680`). The design depends on the monitor changing `vuln`'s PMP while `vuln` waits for a reply, and I needed to know whether S3K allows that before anything else was built. The experiment showed that it does not: a blocked process cannot be reconfigured, a suspended one can, and the resume can itself serve as the reply. It also showed that the socket must be non-yielding. This became the "resume is the reply" protocol.

Fault path. I implemented the memory layout with a stack that has no permanent PMP entry and a pinned trap region (`ab70873`), the boot grants in `app0`, the monitor's main loop, the trap handler that forwards faults, and `fault_handle` with its classification order and attack log. I also ran the baseline experiments in Section 5.

Draft prompt: Add the attack-matrix work and its commits once done.

7.2 Agaton Pirander

Draft prompt: To be written by Agaton, in first person: the enter and exit hooks and the assembly preamble capturing the caller's stack pointer (`66367c1`, `5b910da`); the frame and guard tables with `guard_push`, `guard_pop` and `guard_find` (`f88e39b`); the `-fstack-usage` frame-size table (`7531310`); window selection, slot management and invalidation in `windows.c`.

Draft prompt: Include an AI-assistance disclosure if required by the course, accurately describing its role and how the output was verified.

8 Limitations and Conclusion

8.1 Remaining limitations

The canary seed is built from the timer and an address, which is predictable and not cryptographically strong; in the full design this matters less, since the guard does not rely on the canary being secret. The application-level `__stack_chk_fail` spins instead of terminating. Only `vuln` is protected by guards, and only functions that GCC instruments have canaries. The monitor trusts the frame events, and the guard table has a fixed capacity of 64 frames. Finally, the evaluation is limited to QEMU and one small application.

8.2 Conclusion and future work

Stack canaries can be added to S3K with little effort, but they detect an overflow only after it has happened, and only if the canary is neither skipped nor leaked. Capability-controlled PMP closes most of that gap. By treating the eight PMP entries as a cache over a guard table in a trusted monitor, every live frame's canary and saved return state can be kept unmapped at any call depth, and an overflow faults before it overwrites them. The main constraint is S3K's rule that PMP can only be changed while the target is suspended, which forces one blocking round trip per call, return and window fault. Next steps are to make the failure handler terminate the process, protect the image with separate RX and RW regions, measure the fault and IPC overhead, and reduce the trust placed in frame events.

References

[1] P. Larsen, A. Homescu, S. Brunthaler, and M. Franz, "SoK: Automated Software Diversity," *IEEE Symposium on Security and Privacy*, 2014. doi: 10.1109/SP.2014.25.

[2] L. Litty, H. A. Lagar-Cavilla, and D. Lie, "Hypervisor Support for Identifying Covertly Executing Binaries," *USENIX Security Symposium*, 2008.

[3] T. Abera et al., "C-FLAT: Control-Flow Attestation for Embedded Systems Software," *ACM Conference on Computer and Communications Security*, 2016. doi: 10.1145/2976749.2978358.

[4] S3K project repository, `agpira1/s3k`, branch `2024-sys-sec`, revision `ab70873`. Source snapshot, accessed October 2026.